40 Days of LeetCode Challenge

40 days, 40 problems
DAY :3
LEETCODE PROBLEM NO. 258 : Add Digits
Problem Link : https://leetcode.com/problems/add-digits/

Easy · Digit manipulation · arithmetic operators · loops · Digital Root.
STEP BY STEP SOLUTION:-

Step 1. Understand the question :-

We are given a non- positive integer n, You need to repeatedly add its digits

until the result becomes a single digit number, then return that number.

For example, if num = 38:

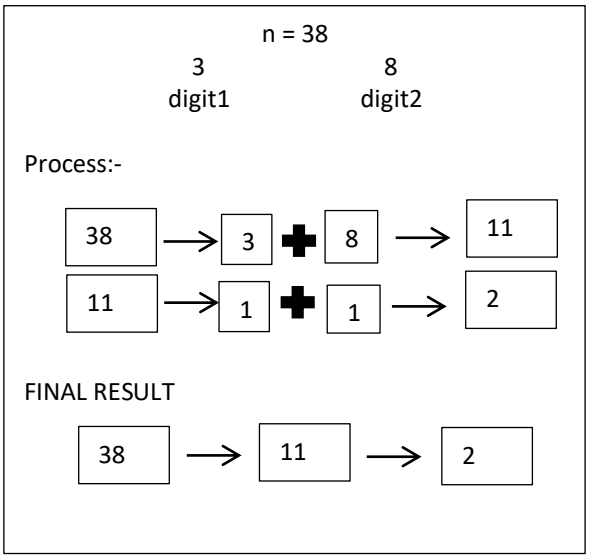
- First, add the digits: (3+8=11)
- The result is still two digits, so add again: (1+1=2)
- Now 2 is a single digit, so return 2.

Step 2. Translating it into simpler language with examples :-

Imagine that every digit in the number is a separate value.

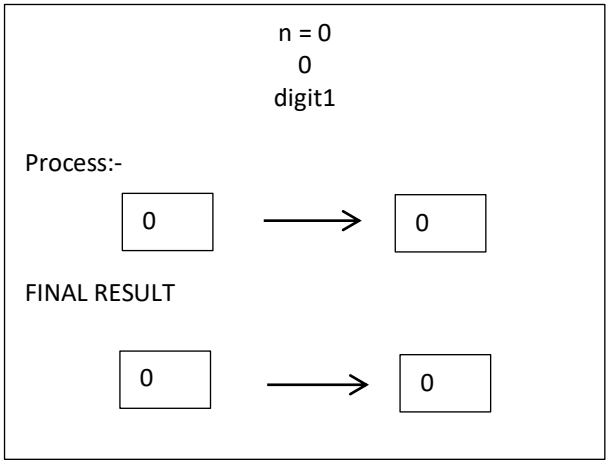
Example 1:
Input: num = 38
Output: 2
Explanation: The process is
38 --> 3 + 8 --> 11
11 --> 1 + 1 --> 2
Since 2 has only one digit, return it.

For the number 38, we have the digits 3 and 8.



Example 2:
Input: num = 0
Explanation: The process is
0 --> 0
Output: 0
Since 0 is only one digit, return it.

For the number 0, has only one digits 0



Step 3. Identify the pattern

Today's problem uses digit manipulation, just like Day 2's LeetCode 1281!

- The main pattern is repeated digit extraction using arithmetic operations.
- Java provides two operators that make this problem straightforward.

A. Modulo operator %

The expression n % 10 returns the last digit of a positive integer.

For example:

- 123 % 10 = 3
- 12 % 10 = 2
- 1 % 10 = 1

B. Integer division /

The expression n / 10 removes the last digit when both operands are integers and n is non-negative.

For example:

- 123 / 10 = 12
- 12 / 10 = 1
- 1 / 10 = 0

Together, these operations allow us to process the digits from right to left without converting the number into a stri ng.

Step 4. Build the logic

Algorithm :-

1. Start.
2. Read the non-negative integer num.
3. Check whether num is greater than or equal to 10.
4. If num is less than 10, proceed directly to Step 10.
5. Initialize a variable sum = 0 to store the sum of the digits.
6. While num > 0, perform the following operations:
    1. Extract the last digit using digit = num % 10.
    2. Add the extracted digit to the sum using sum = sum + digit.
    3. Remove the last digit using integer division: num = num / 10.
7. After processing all digits, assign num = sum.
8. Repeat Steps 3–7 until num becomes less than 10.
9. Exit the loop.
10. Return num as the final single-digit result.
11. Stop.

Pseudocode :-

```
BEGIN
  WHILE num >= 10 DO
    sum ← 0

    WHILE num > 0 DO
      digit ← num MOD 10
      sum ← sum + digit
      num ← num DIV 10
    END WHILE

    num ← sum
  END WHILE

  RETURN num
END
```

NOTE :-

We need two loops in our beginner-friendly solution.

1. The outer while loop checks whether the number still has more than one digit.
2. The inner while loop extracts every digit and adds them together.
3. Once all digits are added, we store the sum back in num.
   We repeat until num becomes a single digit.

Step 5. Code in Java

This is the standard arithmetic solution for LeetCode 258.

```
class Solution {
   public int addDigits(int num) {

      while (num >= 10) {
         int sum = 0;
         while (num > 0) {
            int digit = num % 10;
            sum = sum + digit;
            num = num / 10;
         }
         num = sum;
      }
      return num;
   }
}
```

How to submit

1. Open LeetCode Problem 258 .
2. Select Java as the programming language.
3. Enter the method in the provided Solution class, replacing the existing method if necessary.
4. Click Run to test the examples.
5. Click Submit after the tests pass.

Step 6. Explanation & dry-run

Explanation of the code :-

Code	Why do we need it?
while (num >= 10)	Continues while the number has at least two digits.
int sum = 0;	Starts a fresh digit sum for each round.
while (num > 0)	Processes each digit of the current number.
int digit = num % 10;	Extracts the last digit.
sum = sum + digit;	Adds that digit to the running sum.
num = num / 10;	Removes the last digit using integer division.
num = sum;	Makes the sum the number for the next round.
return num;	Returns the final single digit.

NOTE:-

Sum must be initialized to 0 inside the outer loop, because every round needs a new sum. If we initialize it outside, t he previous round's sum could incorrectly carry over.

Dry run: num = 38

Current num	Digit (num % 10)	Running sum
38	8	8
3	3	11

38 **11** **2**



Original After round 1 Final answer

- Extract 1 → sum = 1
- Extract 1 → sum = 2
    - Assign num = 2. Since 2 < 10, the outer loop stops.
    - Returned output = 2

7. Analyze complexity and optimization

Time complexity: (O(log n))

The number of digits is proportional to (logn), and the repeated digit sums take linear time in the total nu mber of digits processed.

Auxiliary space complexity: (O(1))

We use only a few integer variables, regardless of the input size.

NOTE:-

Can it be optimized further?

LeetCode 258 has a mathematical trick called the digital root. For any nonnegative integer, the final single-digit result can be calculated directly:

$$\text{digitalRoot}(n) = \begin{cases} 0 & n=0 \\ 1+(n-1) \bmod 9 & n>0 \end{cases}$$

This gives us an (O(1)) time and (O(1)) space solution

Code in Java :-

```
  public int addDigits(int num) {
    if (num == 0) {
      return 0;
    }

    return 1 + (num - 1) % 9;
  }
}
```

NOTE:-

Why does this work?

In base 10, a number has the same remainder modulo 9 as the sum of its digits. Repeating digit addition preserves that remainder, and the formula handles the special case of multiples of 9 correctly.

For num = 38:

- (38-1=37)
- (37 mod 9=1)
- (1+1=2)

OR:-

```
class Solution {
   public int addDigits(int num) {
      return num == 0 ? 0 : (num % 9 == 0 ? 9 : num % 9);
   }
}
```

SCREENSHOT:-

